Web Engineering & Development (SWE 363)

Web Security

In today's lecture:

- Third-party web APIs
- Token-based user authentication
- Password hashing

Reference:

- Zybook: 8.1-8.3

Third-party web APIs

What is a Third-Party Web API?

API = Application Programming Interface

A **third-party web API** is a service provided by another company that your application can use

Key idea:

- You don't need to build everything yourself
- Use existing services to add features to your app
- Access data and services over the internet

Examples of Third-Party APIs

Weather APIs:

- Get current weather, forecasts, historical data

Social Media APIs:

- Twitter, Facebook, Instagram integration

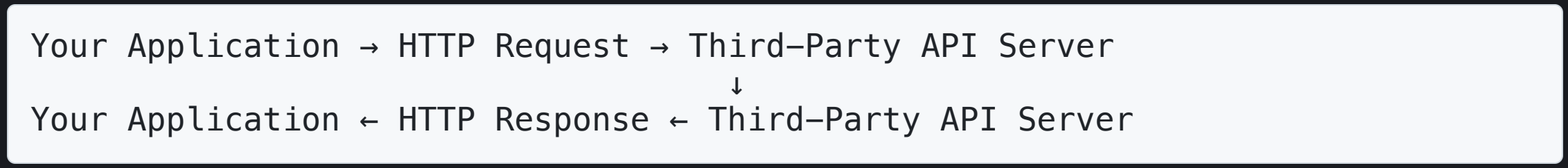
Payment APIs:

- Stripe, PayPal for processing payments

Maps APIs:

- Google Maps, Mapbox for location services

How Third-Party APIs Work



Flow:

1. Your app sends a request to the API
2. API processes the request
3. API sends back data (usually JSON)
4. Your app uses the data

Making API Requests with fetch()

The `fetch()` function is built into JavaScript for making HTTP requests

Basic syntax:

```
fetch(url)
  .then(response => response.json())
  .then(data => {
    // Use the data
  })
  .catch(error => {
    // Handle errors
  });
```

Using fetch() with async/await

Modern approach using async/await:

```
async function getWeather(city) {
  try {
    const response = await fetch(
      `https://goweather.herokuapp.com/weather/${city}`
    );
    const data = await response.json();
    return data;
  } catch (error) {
    console.error("Error fetching weather:", error);
  }
}
```

Handling API Responses

Check if response is successful:

```
if (!response.ok) {
  // Handle error
  return res.status(500).json({ error: "API error" });
}
```

Parse JSON data:

```
const data = await response.json();
```

Use the data:

```
res.json({ city: city, temp: data.temperature });
```

URL Encoding

Why encode URLs?

- City names may contain special characters
- Spaces, Arabic characters, etc. need encoding

How to encode:

```
const city = "New York";
const encoded = encodeURIComponent(city);
// Result: "New%20York"

const url = `https://api.example.com/weather/${encoded}`;
```

Always encode user input in URLs!

Error Handling for APIs

Common errors:

- API server is down
- Invalid request parameters
- Network connection issues
- Rate limiting (too many requests)

Error Handling for APIs

Best practice:

```
try {
  const response = await fetch(url);
  if (!response.ok) {
    return res.status(500).json({ error: "API error" });
  }
  const data = await response.json();
  res.json(data);
} catch (error) {
  res.status(500).json({ error: "Server error" });
}
```

Benefits of Using Third-Party APIs

Advantages:

- Save development time
- Access to specialized data/services
- Regular updates and maintenance by provider
- Focus on your core application features

Considerations:

- API may change or become unavailable
- May have usage limits or costs
- Dependency on external service

Token-based User Authentication

What is Authentication?

Authentication = Verifying who a user is

Why we need it:

- Protect sensitive resources
- Ensure only authorized users access data
- Track user actions
- Personalize user experience

Example:

- Only logged-in users can view their profile
- Only authenticated users can access protected routes

Authentication Methods

Common authentication approaches:

1. Session-Based Authentication:

- Server creates session after login
- Session ID stored in cookie
- Server maintains session state
- Traditional approach, widely used

2. Token-Based Authentication:

- Server creates token after login
- Token sent with each request
- Server doesn't store session state
- Modern approach, scalable

Authentication Methods

Common authentication approaches:

3. API Key Authentication:

- Simple key for API access
- Used for service-to-service communication
- Less secure for user authentication

Traditional Authentication Problem

Session-based authentication:

- Server stores session information
- Requires server to remember each user
- Difficult to scale across multiple servers
- Server must maintain session state

Problem:

- What if you have multiple servers?
- What if server restarts?
- How to share sessions across servers?

Token-Based Authentication

Solution: Token-based authentication

How it works:

1. User logs in with credentials
2. Server verifies credentials
3. Server creates a token and sends it to client
4. Client sends token with each request
5. Server verifies token to authenticate user

Key advantage: Stateless - server doesn't store session data

What is a JWT?

JWT = JSON Web Token

A **JWT** is a compact, URL-safe token that contains information about a user

Structure:

```
header.payload.signature
```

Example:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6InVzZXJAZXhhbXBsZS5jb20ifQ.signature
```

JWT Structure (Three parts)

1. Header:

- Describes the token type and encryption method
- Example: `{"alg": "HS256", "typ": "JWT"}`

2. Payload:

- Contains user information (claims)
- Example: `{"email": "user@example.com", "exp": 1234567890}`

3. Signature:

- Verifies token authenticity
- Created using secret key

JWT Advantages

Stateless:

- No session storage needed
- Server doesn't need to remember users

Scalable:

- Works across multiple servers
- No shared session storage required

Compact:

- Small size, easy to send in HTTP headers
- Can be stored in cookies

Secure:

- Digitally signed
- Can expire automatically

Installing JWT Library

Install jsonwebtoken:

```
npm install jsonwebtoken
```

Import in your server:

```
const jwt = require("jsonwebtoken");
```

Set a secret key:

```
const JWT_SECRET = "your-secret-key-here";
// In production, use environment variables!
```

Creating a JWT Token

After successful login:

```
const token = jwt.sign(
  { email: user.email },  // Payload (user data)
  JWT_SECRET,             // Secret key
  { expiresIn: "1h" }     // Expiration time
);

res.json({ token });
```

Token contains:

- User email (or other identifying info)
- Expiration time (1 hour)
- Digital signature

Sending Token in Requests

Client sends token in Authorization header:

```
Authorization: Bearer <token>
```

Important:

- Must include the word "Bearer"
- One space between "Bearer" and token
- No quotes around the token

Example:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Verifying Token on Server

Extract and verify token:

```
app.get("/protected-route", (req, res) => {
  // 1. Get Authorization header
  const auth = req.headers.authorization;

  // 2. Extract token (remove "Bearer ")
  const token = auth.split(" ")[1];

  // 3. Verify token
  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    // Token is valid, user is authenticated
    res.json({ message: "Access granted", user: decoded });
  } catch (error) {
    return res.status(401).json({ error: "Invalid token" });
  } });
```

Complete Login Flow

1. User sends credentials:

```
POST /login
Body: { "email": "user@example.com", "password": "password123" }
```

2. Server verifies credentials:

```
const user = users.find(u => u.email === email);
const match = await bcrypt.compare(password, user.passwordHash);
```

3. Server creates token:

```
const token = jwt.sign({ email }, JWT_SECRET, { expiresIn: "1h" });
res.json({ token });
```

Using Token for Protected Routes

4. Client stores token:

```
// Save token from login response
localStorage.setItem("token", response.token);
```

5. Client sends token with requests:

```
fetch("/protected-route", {
  headers: { "Authorization": `Bearer ${token}` }
});
```

6. Server verifies token:

```
const decoded = jwt.verify(token, JWT_SECRET); // User is authenticated!
```

Token Expiration

Why tokens expire:

- Security: Limits damage if token is stolen
- Forces re-authentication periodically
- Reduces risk of unauthorized access

Setting expiration:

```
jwt.sign(
  { email },
  JWT_SECRET,
  { expiresIn: "1h" }  // Expires in 1 hour
);
```

Handling Expired Tokens

When token expires:

```
try { const decoded = jwt.verify(token, JWT_SECRET);
} catch (error) {
  if (error.name === "TokenExpiredError") {
    return res.status(401).json({ error: "Token expired" });
  }
  return res.status(401).json({ error: "Invalid token" });
}
```

Client response:

- Redirect to login page
- Request new token
- Show appropriate error message

Password Hashing

Why Hash Passwords?

NEVER store passwords in plain text!

If database is compromised:

- Plain text: Attackers can read all passwords immediately
- Hashed: Attackers see random strings, not actual passwords

Real-world example:

- Many companies have been hacked
- If passwords were hashed, damage is limited
- If passwords were plain text, all accounts are compromised

What is Password Hashing?

Hashing = Converting password into a fixed-length string

Properties:

- **One-way:** Cannot reverse hash to get original password
- **Deterministic:** Same password always produces same hash
- **Unique:** Different passwords produce different hashes

Example:

```
Password: "mypassword123"
Hash:     "$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy"
```

Hash Functions

Simple hash (NOT secure for passwords):

```
// MD5, SHA-1 - Fast but insecure
// Can be cracked quickly with rainbow tables
```

Secure hash for passwords:

```
// bcrypt - Slow by design, secure
// Designed specifically for password hashing
```

Why slow is good:

- Makes brute force attacks impractical
- Each attempt takes time and attackers cannot quickly try many passwords

What is Salt?

Salt = Random data added to password before hashing

Why salt is important:

- Same password produces different hashes
- Prevents rainbow table attacks
- Makes each hash unique

Example:

```
Password: "password123"
Salt:     "randomSalt123"
Hash:     hash(password + salt)
```

What is Salt?

Example:

```
Password: "password123"
Salt:     "randomSalt123"
Hash:     hash(password + salt)
```

Without salt:

- Two users with same password have same hash
- Attackers can identify common passwords

bcrypt Library

bcrypt = Popular password hashing library

Features:

- Automatically generates salt
- Configurable cost factor (how slow)
- Secure and widely used
- Built for password hashing

bcrypt Library

Install:

```
npm install bcryptjs
```

Import:

```
const bcrypt = require("bcryptjs");
```

Hashing Passwords with bcrypt

During user registration:

```
const password = req.body.password;
const saltRounds = 10;  // Cost factor (higher = slower but more secure)

const hash = await bcrypt.hash(password, saltRounds);

// Store hash in database
users.push({
  email: email,
  passwordHash: hash  // Store hash, NOT plain password!
});
```

Important: Never store the original password!

Comparing Passwords with bcrypt

During user login:

```
const { email, password } = req.body;

// Find user
const user = users.find(u => u.email === email);

// Compare password with stored hash
const match = await bcrypt.compare(password, user.passwordHash);

if (match) {
  // Password is correct
  // Create token and send to client
} else {
  // Password is incorrect
  res.status(400).json({ error: "Wrong password" });
}
```

bcrypt.compare() Function

How it works:

```
bcrypt.compare(plainPassword, hashedPassword)
```

Process:

1. Takes plain password from user
2. Takes stored hash from database
3. Hashes plain password with same salt
4. Compares the two hashes
5. Returns `true` if they match, `false` otherwise

You don't need to extract salt manually!

Password Security Best Practices

1. **Always hash passwords:** Never store plain text passwords
2. **Use strong hashing:** Use bcrypt or similar (not MD5, SHA-1)
3. **Use appropriate cost factor:**
   - Balance between security and performance
   - Typically 10-12 rounds
4. **Validate password strength:**
   - Minimum length
   - Require special characters
   - Check against common passwords

Password Cracking Attacks

Dictionary Attack:

- Try common passwords from a list
- "password", "123456", "qwerty"

Rainbow Tables:

- Pre-computed hash tables
- Salt prevents this attack

Brute Force:

- Try every possible combination
- Slow hashing makes this impractical

Protection:

- Strong passwords
- Password hashing with salt
- Rate limiting login attempts

Complete Registration Flow

1. User submits registration:

```
POST /register
Body: { "email": "user@example.com", "password": "mypassword123" }
```

2. Server validates input:

```
if (!email || !password) {
  return res.status(400).json({ error: "Email and password required" });
}
```

Complete Registration Flow

3. Server hashes password:

```
const hash = await bcrypt.hash(password, 10);
```

4. Server stores user:

```
users.push({ email, passwordHash: hash });
res.status(201).json({ message: "User registered!" });
```

Complete Login Flow with Hashing

1. User submits login:

```
POST /login
Body: { "email": "user@example.com", "password": "mypassword123" }
```

2. Server finds user:

```
const user = users.find(u => u.email === email);
if (!user) {
  return res.status(400).json({ error: "User not found" });
}
```

Complete Login Flow with Hashing

3. Server compares password:

```
const match = await bcrypt.compare(password, user.passwordHash);
if (!match) {
  return res.status(400).json({ error: "Wrong password" });
}
```

4. Server creates token:

```
const token = jwt.sign({ email }, JWT_SECRET, { expiresIn: "1h" });
res.json({ token });
```

Security Summary

Three layers of security:

1. Password Hashing: Protects passwords if database is compromised. Use bcrypt with salt.

2. Token Authentication:

- Verifies user identity for each request
- Stateless and scalable

3. Protected Routes:

- Only authenticated users can access
- Token must be valid and not expired

Common Mistakes to Avoid

Password mistakes:

- Storing plain text passwords
- Using weak hashing (MD5, SHA-1)
- Not using salt
- Storing salt separately from hash

Token mistakes:

- Not checking token expiration
- Using weak secret keys
- Not validating token on every request
- Storing tokens insecurely on client

30m

Demo

8.1 Third-Party APIs and Authentication